CONTROL AND OBSERVABILITY

Intercept and control agent behavior with hooks

Intercept and customize agent behavior at key execution points with hooks

Hooks are callback functions that run your code in response to agent events, like a tool being called, a session starting, or execution stopping. With hooks, you can:

- **Block dangerous operations** before they execute, like destructive shell commands or unauthorized file access
- **Log and audit** every tool call for compliance, debugging, or analytics
- **Transform inputs and outputs** to sanitize data, inject credentials, or redirect file paths
- **Require human approval** for sensitive actions like database writes or API calls
- **Track session lifecycle** to manage state, clean up resources, or send notifications

This guide covers how hooks work and how to configure them, with examples for common patterns like blocking tools, modifying inputs, and forwarding notifications.

How hooks work

1 **An event fires**

Something happens during agent execution and the SDK fires an event: a tool is about to be called (`PreToolUse`), a tool returned a result (`PostToolUse`), a subagent started or stopped, the agent is idle, or execution finished. See the **full list of events**.

2 **The SDK collects registered hooks**

The SDK checks for hooks registered for that event type. This includes callback hooks you pass in `options.hooks` and shell command hooks from settings files when the corresponding `settingSources` or `setting_sources` entry is enabled, which it is for default `query()` options.

3 **Matchers filter which hooks run**

If a hook has a `matcher` pattern (like `"Write|Edit"`), the SDK tests it against the event's target (for example, the tool name). Hooks without a matcher run for every event of that type.

4 **Callback functions execute**

Each matching hook's **callback function** receives input about what's happening: the tool name, its arguments, the session ID, and other event-specific details.

5 **Your callback returns a decision**

After performing any operations (logging, API calls, validation), your callback returns an **output object** that tells the agent what to do: allow the operation, block it, modify the input, or inject context into the conversation.

The following example puts these steps together. It registers a `PreToolUse` hook (step 1) with a `"Write|Edit"` matcher (step 3) so the callback only fires for file-writing tools. When triggered, the callback receives the tool's input (step 4), checks if the file path targets a `.env` file, and returns `permissionDecision: "deny"` to block the operation (step 5):

```python
import asyncio
from claude_agent_sdk import (
    AssistantMessage,
    ClaudeSDKClient,
    ClaudeAgentOptions,
    HookMatcher,
    ResultMessage,
)


# Define a hook callback that receives tool call details
async def protect_env_files(input_data, tool_use_id, context):
    # Extract the file path from the tool's input arguments
    file_path = input_data["tool_input"].get("file_path", "")
    file_name = file_path.split("/")[-1]

    # Block the operation if targeting a .env file
    if file_name == ".env":
        return {
            "hookSpecificOutput": {
                "hookEventName": input_data["hook_event_name"],
                "permissionDecision": "deny",
                "permissionDecisionReason": "Cannot modify .env files",
            }
        }

    # Return empty object to allow the operation
    return {}


async def main():
    options = ClaudeAgentOptions(
        hooks={
            # Register the hook for PreToolUse events
            # The matcher filters to only Write and Edit tool calls
            "PreToolUse": [HookMatcher(matcher="Write|Edit", hooks=
[protect_env_files])]
        }
    )

    async with ClaudeSDKClient(options=options) as client:
        await client.query("Update the database configuration")
        async for message in client.receive_response():
            # Filter for assistant and result messages
```

```
            if isinstance(message, (AssistantMessage, ResultMessage)):
                print(message)


asyncio.run(main())
```

Available hooks

The SDK provides hooks for different stages of agent execution. Some hooks are available in both SDKs, while others are TypeScript-only.

Hook Event	Python SDK	TypeScript SDK	What triggers it	Example use case
`PreToolUse`	Yes	Yes	Tool call request (can block or modify)	Block dangerous shell commands
`PostToolUse`	Yes	Yes	Tool execution result	Log all file changes to audit trail
`PostToolUseFailure`	Yes	Yes	Tool execution failure	Handle or log tool errors
`PostToolBatch`	No	Yes	A full batch of tool calls resolves, once per batch before the next model call	Inject conventions once for the whole batch
`UserPromptSubmit`	Yes	Yes	User prompt submission	Inject additional context into prompts
`MessageDisplay`	No	Yes	An assistant message with text completes, once per message with the full message text	Redact or reformat the displayed text without changing the transcript
`Stop`	Yes	Yes	Agent execution stop	Save session state before exit
`SubagentStart`	Yes	Yes	Subagent initialization	Track parallel task spawning
`SubagentStop`	Yes	Yes	Subagent completion	Aggregate results from parallel tasks
`PreCompact`	Yes	Yes	Conversation compaction request	Archive full transcript before summarizing
`PermissionRequest`	Yes	Yes	Permission dialog would be displayed	Custom permission handling

Hook Event	Python SDK	TypeScript SDK	What triggers it	Example use case
`SessionStart`	No	Yes	Session initialization	Initialize logging and telemetry
`SessionEnd`	No	Yes	Session termination	Clean up temporary resources
`Notification`	Yes	Yes	Agent status messages	Send agent status updates to Slack or PagerDuty
`Setup`	No	Yes	Session setup/maintenance	Run initialization tasks
`TeammateIdle`	No	Yes	Teammate becomes idle	Reassign work or notify
`TaskCompleted`	No	Yes	Background task completes	Aggregate results from parallel tasks
`ConfigChange`	No	Yes	Configuration file changes	Reload settings dynamically
`WorktreeCreate`	No	Yes	Git worktree created	Track isolated workspaces
`WorktreeRemove`	No	Yes	Git worktree removed	Clean up workspace resources

Configure hooks

To configure a hook, pass it in the `hooks` field of your agent options (`ClaudeAgentOptions` in Python, the `options` object in TypeScript):

```
options = ClaudeAgentOptions(
    hooks={"PreToolUse": [HookMatcher(matcher="Bash", hooks=[my_callback])]}
)

async with ClaudeSDKClient(options=options) as client:
    await client.query("Your prompt")
    async for message in client.receive_response():
        print(message)
```

The `hooks` option is a dictionary in Python or an object in TypeScript, where:

- **Keys**: **hook event names** such as `'PreToolUse'`, `'PostToolUse'`, and `'Stop'`

- **Values**: arrays of **matchers**, each containing an optional filter pattern and your **callback functions**

Matchers

Use matchers to filter when your callbacks fire. The `matcher` field matches against a different value depending on the hook event type. For example, tool-based hooks match against the tool name, while `Notification` hooks match against the notification type. See the **Claude Code hooks reference** for the full list of matcher values for each event type.

SDK matchers follow the same rules as **matchers in settings files**: a matcher containing only letters, digits, `_`, spaces, `,`, and `|` is compared as an exact string, with alternatives separated by `|` or `,` and optional surrounding whitespace, so `Write|Edit` and `Write, Edit` each match exactly those two tools. A matcher of `*`, an empty string, or omitting the matcher entirely matches every occurrence of the event; a matcher containing any other character is evaluated as a regular expression, so `^mcp__` matches every MCP tool. A matcher like `mcp__memory` contains only letters and underscores, so it is compared as an exact string and matches no tool; use `mcp__memory__.*` to match every tool from that server.

Option	Type	Default	Description
`matcher`	`string`	`undefined`	Pattern matched against the event's filter field, following the comparison rules above. For tool hooks, this is the tool name. Built-in tools include `Bash`, `Read`, `Write`, `Edit`, `Glob`, `Grep`, `WebFetch`, `Agent`, and others (see **Tool Input Types** for the full list). MCP tools use the pattern `mcp__<server>__<action>`.
`hooks`	`HookCallback[]`	-	Required. Array of callback functions to execute when the pattern matches
`timeout`	`number`	`60`	Timeout in seconds

Use the `matcher` pattern to target specific tools whenever possible. A matcher with `'Bash'` only runs for Bash commands, while omitting the pattern runs your callbacks for every occurrence of the event.

For tool-based hooks, matchers only filter by tool name, not by file paths or other arguments. To filter by file path, check `tool_input.file_path` inside your callback.

Discovering tool names: See **Tool Input Types** for the full list of built-in tool names, or add a hook without a matcher to log all tool calls your session makes.

MCP tool naming: MCP tools always start with `mcp__` followed by the server name and action: `mcp__<server>__<action>`. For example, if you configure a server named `playwright`, its tools are named `mcp__playwright__browser_screenshot`, `mcp__playwright__browser_click`, and so on. The server name comes from the key you use in the `mcpServers` configuration.

Callback functions

Inputs

Every hook callback receives three arguments:

- **Input data:** a typed object containing event details. Each hook type has its own input shape. For example, `PreToolUseHookInput` includes `tool_name` and `tool_input`, while `NotificationHookInput` includes `message`. See the full type definitions in the **TypeScript** and **Python** SDK references.
  - All hook inputs share `session_id`, `cwd`, and `hook_event_name`.
  - `agent_id` and `agent_type` are populated when the hook fires inside a subagent. In TypeScript, these are on the base hook input and available to all hook types. In Python, they are on `PreToolUse`, `PostToolUse`, and `PostToolUseFailure` only.
- **Tool use ID** (`str | None` / `string | undefined`): correlates `PreToolUse` and `PostToolUse` events for the same tool call.
- **Context:** in TypeScript, contains a `signal` property (`AbortSignal`) for cancellation. In Python, this argument is reserved for future use.

Outputs

Your callback returns an object with two categories of fields:

- **Top-level fields** work the same on every event: `systemMessage` shows a message to the user, and `continue` (`continue_` in Python) determines whether the agent keeps running after this hook.
- `hookSpecificOutput` controls the current operation. The fields inside depend on the hook event type. For `PreToolUse` hooks, this is where you set `permissionDecision` (`"allow"`, `"deny"`, `"ask"`, or `"defer"`), `permissionDecisionReason`, and `updatedInput`. Returning `"defer"` ends the query so you can **resume it later**. For `PostToolUse` hooks, you can set `additionalContext` to append information to the tool result. To replace the tool's output before

Claude sees it, set `updatedToolOutput`, which works for any tool in both SDKs. The older `updatedMCPToolOutput` field replaces MCP tool output only and is deprecated.

Return `{}` to allow the operation without changes. SDK callback hooks use the same JSON output format as **Claude Code shell command hooks**, which documents every field and event-specific option. For the SDK type definitions, see the **TypeScript** and **Python** SDK references.

> When multiple hooks or permission rules apply, `deny` takes priority over `defer`, which takes priority over `ask`, which takes priority over `allow`. If any hook returns `deny`, the operation is blocked regardless of other hooks.

Asynchronous output

By default, the agent waits for your hook to return before proceeding. If your hook performs a side effect, such as logging or sending a webhook, and doesn't need to influence the agent's behavior, you can return an async output instead. This tells the agent to continue immediately without waiting for the hook to finish:

```
async def async_hook(input_data, tool_use_id, context):
    # Start a background task, then return immediately
    asyncio.create_task(send_to_logging_service(input_data))
    return {"async_": True, "asyncTimeout": 30000}
```

Field	Type	Description
`async`	`true`	Signals async mode. The agent proceeds without waiting. In Python, use `async_` to avoid the reserved keyword.
`asyncTimeout`	`number`	Optional timeout in milliseconds for the background operation

> Async outputs can't block, modify, or inject context into the operation since the agent has already moved on. Use them only for side effects like logging, metrics, or notifications.

Examples

Modify tool input

This example intercepts Write tool calls and rewrites the `file_path` argument to prepend `/sandbox`, redirecting all file writes to a sandboxed directory. The callback returns `updatedInput`

with the modified path and `permissionDecision: 'allow'` to auto-approve the rewritten operation:

```
async def redirect_to_sandbox(input_data, tool_use_id, context):
    if input_data["hook_event_name"] != "PreToolUse":
        return {}

    if input_data["tool_name"] == "Write":
        original_path = input_data["tool_input"].get("file_path", "")
        return {
            "hookSpecificOutput": {
                "hookEventName": input_data["hook_event_name"],
                "permissionDecision": "allow",
                "updatedInput": {
                    **input_data["tool_input"],
                    "file_path": f"/sandbox{original_path}",
                },
            }
        }
    return {}
```

When using `updatedInput`, you must also include `permissionDecision: 'allow'` to auto-approve the modified input or `permissionDecision: 'ask'` to show it to the user. With `'defer'`, `updatedInput` is ignored. Always return a new object rather than mutating the original `tool_input`.

Add context and block a tool

This example blocks writes to the `/etc` directory and explains why to both the model and the user:

- `permissionDecision: 'deny'` stops the tool call.
- `permissionDecisionReason` tells the model why, so it avoids retrying.
- `systemMessage` shows the user what happened.

```
async def block_etc_writes(input_data, tool_use_id, context):
    file_path = input_data["tool_input"].get("file_path", "")

    if file_path.startswith("/etc"):
        return {
            # Top-level field: message shown to the user
            "systemMessage": "Remember: system directories like /etc are
protected.",
            # hookSpecificOutput: block the operation
            "hookSpecificOutput": {
                "hookEventName": input_data["hook_event_name"],
                "permissionDecision": "deny",
                "permissionDecisionReason": "Writing to /etc is not allowed",
            },
        }
    return {}
```

Auto-approve specific tools

By default, the agent may prompt for permission before using certain tools. This example auto-approves read-only filesystem tools (Read, Glob, Grep) by returning `permissionDecision: 'allow'`, letting them run without user confirmation while leaving all other tools subject to normal permission checks:

```
async def auto_approve_read_only(input_data, tool_use_id, context):
    if input_data["hook_event_name"] != "PreToolUse":
        return {}

    read_only_tools = ["Read", "Glob", "Grep"]
    if input_data["tool_name"] in read_only_tools:
        return {
            "hookSpecificOutput": {
                "hookEventName": input_data["hook_event_name"],
                "permissionDecision": "allow",
                "permissionDecisionReason": "Read-only tool auto-approved",
            }
        }
    return {}
```

Register multiple hooks

When an event fires, all matching hooks run in parallel. For permission decisions, the most restrictive result applies: a single `deny` blocks the tool call regardless of what the other hooks return. Because completion order is non-deterministic, write each hook to act independently rather than relying on another hook having run first.

The example below registers three independent checks for every tool call:

```
options = ClaudeAgentOptions(
    hooks={
        "PreToolUse": [
            HookMatcher(hooks=[authorization_check]),
            HookMatcher(hooks=[input_validator]),
            HookMatcher(hooks=[audit_logger]),
        ]
    }
)
```

Filter with multi-tool matchers

Use multi-tool matchers to share one callback across related tools. This example registers three matchers with different scopes:

- A pipe-separated exact list (`Write|Edit|Delete`) triggers `file_security_hook` only for file modification tools.
- A regex (`^mcp__`) triggers `mcp_audit_hook` for any MCP tool whose name starts with `mcp__` .
- An omitted matcher triggers `global_logger` for every tool call regardless of name.

```
options = ClaudeAgentOptions(
    hooks={
        "PreToolUse": [
            # Match file modification tools
            HookMatcher(matcher="Write|Edit|Delete", hooks=
[file_security_hook]),
            # Match all MCP tools
            HookMatcher(matcher="^mcp__", hooks=[mcp_audit_hook]),
            # Match everything (no matcher)
            HookMatcher(hooks=[global_logger]),
        ]
    }
)
```

Track subagent activity

Use `SubagentStop` hooks to monitor when subagents finish their work. See the full input type in the **TypeScript** and **Python** SDK references. This example logs a summary each time a subagent completes:

```
async def subagent_tracker(input_data, tool_use_id, context):
    # Log subagent details when it finishes
    print(f"[SUBAGENT] Completed: {input_data['agent_id']}")
    print(f"  Transcript: {input_data['agent_transcript_path']}")
    print(f"  Tool use ID: {tool_use_id}")
    print(f"  Stop hook active: {input_data.get('stop_hook_active')}")
    return {}


options = ClaudeAgentOptions(
    hooks={"SubagentStop": [HookMatcher(hooks=[subagent_tracker])]}
)
```

Make HTTP requests from hooks

Hooks can perform asynchronous operations like HTTP requests. Catch errors inside your hook instead of letting them propagate, since an unhandled exception can interrupt the agent.

This example sends a webhook after each tool completes, logging which tool ran and when. The hook catches errors so a failed webhook doesn't interrupt the agent:

```python
import asyncio
import json
import urllib.request
from datetime import datetime


def _send_webhook(tool_name):
    """Synchronous helper that POSTs tool usage data to an external
webhook."""
    data = json.dumps(
        {
            "tool": tool_name,
            "timestamp": datetime.now().isoformat(),
        }
    ).encode()
    req = urllib.request.Request(
        "https://api.example.com/webhook",
        data=data,
        headers={"Content-Type": "application/json"},
        method="POST",
    )
    urllib.request.urlopen(req)


async def webhook_notifier(input_data, tool_use_id, context):
    # Only fire after a tool completes (PostToolUse), not before
    if input_data["hook_event_name"] != "PostToolUse":
        return {}

    try:
        # Run the blocking HTTP call in a thread to avoid blocking the event
loop
        await asyncio.to_thread(_send_webhook, input_data["tool_name"])
    except Exception as e:
        # Log the error but don't raise. A failed webhook shouldn't stop the
agent
        print(f"Webhook request failed: {e}")

    return {}
```

Forward notifications to Slack

Use `Notification` hooks to receive system notifications from the agent and forward them to

external services. Notifications fire for event types such as:

- `permission_prompt` when Claude needs permission
- `idle_prompt` when Claude is waiting for input
- `auth_success` when authentication completes
- `elicitation_dialog`, `elicitation_complete`, and `elicitation_response` for user-prompt elicitation flows

Each notification includes a `message` field with a human-readable description and optionally a `title`.

This example forwards every notification to a Slack channel. It requires a **Slack incoming webhook URL**, which you create by adding an app to your Slack workspace and enabling incoming webhooks:

```python
import asyncio
import json
import urllib.request

from claude_agent_sdk import ClaudeSDKClient, ClaudeAgentOptions, HookMatcher


def _send_slack_notification(message):
    """Synchronous helper that sends a message to Slack via incoming
webhook."""
    data = json.dumps({"text": f"Agent status: {message}"}).encode()
    req = urllib.request.Request(
        "https://hooks.slack.com/services/YOUR/WEBHOOK/URL",
        data=data,
        headers={"Content-Type": "application/json"},
        method="POST",
    )
    urllib.request.urlopen(req)


async def notification_handler(input_data, tool_use_id, context):
    try:
        # Run the blocking HTTP call in a thread to avoid blocking the event
loop
        await asyncio.to_thread(_send_slack_notification,
input_data.get("message", ""))
    except Exception as e:
        print(f"Failed to send notification: {e}")

    # Return empty object. Notification hooks don't modify agent behavior
    return {}


async def main():
    options = ClaudeAgentOptions(
        hooks={
            # Register the hook for Notification events (no matcher needed)
            "Notification": [HookMatcher(hooks=[notification_handler])],
        },
    )

    async with ClaudeSDKClient(options=options) as client:
        await client.query("Analyze this codebase")
        async for message in client.receive_response():
```

```
            print(message)


asyncio.run(main())
```

Fix common issues

Hook not firing

- Verify the hook event name is correct and case-sensitive (`PreToolUse` , not `preToolUse`)
- Check that your matcher pattern matches the tool name exactly
- Ensure the hook is under the correct event type in `options.hooks`
- For non-tool hooks like `Stop` and `SubagentStop` , matchers match against different fields (see **matcher patterns**)
- Hooks may not fire when the agent hits the `max_turns` limit because the session ends before hooks can execute

Matcher not filtering as expected

Matchers only match tool names, not file paths or other arguments. To filter by file path, check `tool_input.file_path` inside your hook:

```
const myHook: HookCallback = async (input, toolUseID, { signal })
=> {
  const preInput = input as PreToolUseHookInput;
  const toolInput = preInput.tool_input as Record<string,
unknown>;
  const filePath = toolInput?.file_path as string;
  if (!filePath?.endsWith(".md")) return {}; // Skip non-markdown
files
  // Process markdown files...
  return {};
};
```

Hook timeout

- Increase the `timeout` value in the `HookMatcher` configuration
- Use the `AbortSignal` from the third callback argument to handle cancellation gracefully in

TypeScript

Tool blocked unexpectedly

- Check all `PreToolUse` hooks for `permissionDecision: 'deny'` returns
- Add logging to your hooks to see what `permissionDecisionReason` they're returning
- Verify matcher patterns aren't too broad: an empty matcher matches all tools

Modified input not applied

- Ensure `updatedInput` is inside `hookSpecificOutput`, not at the top level:

```
return {
  hookSpecificOutput: {
    hookEventName: "PreToolUse",
    permissionDecision: "allow",
    updatedInput: { command: "new command" }
  }
};
```

- Return `permissionDecision: 'allow'` to auto-approve the modified input, or `'ask'` to show it to the user for approval
- Include `hookEventName` in `hookSpecificOutput` to identify which hook type the output is for

Session hooks not available in Python

`SessionStart` and `SessionEnd` can be registered as SDK callback hooks in TypeScript, but aren't available in the Python SDK because its `HookEvent` type omits them. In Python, they are only available as **shell command hooks** defined in settings files such as `.claude/settings.json`. To load shell command hooks from your SDK application, include the appropriate setting source with `setting_sources` or `settingSources`:

```
options = ClaudeAgentOptions(
    setting_sources=["project"],  # Loads .claude/settings.json including
hooks
)
```

To run initialization logic as a Python SDK callback instead, use the first message from `client.receive_response()` as your trigger.

Subagent permission prompts multiplying

When spawning multiple subagents, each one may request permissions separately. Subagents don't automatically inherit parent agent permissions. To avoid repeated prompts, use `PreToolUse` hooks to auto-approve specific tools, or configure permission rules that apply to subagent sessions.

Recursive hook loops with subagents

A `UserPromptSubmit` hook that spawns subagents can create infinite loops if those subagents trigger the same hook. To prevent this:

- Check for a subagent indicator in the hook input before spawning
- Use a shared variable or session state to track whether you're already inside a subagent
- Scope hooks to only run for the top-level agent session

systemMessage not appearing in output

The `systemMessage` field shows a message to the user, not the model. By default the SDK doesn't surface hook output in the message stream, so the message may not appear unless you set `includeHookEvents` (`include_hook_events` in Python). To pass context to the model instead, return `additionalContext`.

If you need to surface hook decisions to your application reliably, log them separately or use a dedicated output channel.

Related resources

- **Claude Code hooks reference**: full JSON input/output schemas, event documentation, and matcher patterns
- **Claude Code hooks guide**: shell command hook examples and walkthroughs
- **TypeScript SDK reference**: hook types, input/output definitions, and configuration options
- **Python SDK reference**: hook types, input/output definitions, and configuration options
- **Permissions**: control what your agent can do

- **Custom tools**: build tools to extend agent capabilities